

# Assignment 8 - Multi-Agent Growing-Graph Orchestrator

This repository contains Session 8 (S8) of the EAGV3 course: a multi-agent AI system where a directed graph of skills is the agent loop. Each node is a specialized LLM-backed role (Planner, Researcher, Coder, Formatter, etc.); edges carry typed results between nodes; and the graph grows at runtime as skills finish and the orchestrator splices in successors.

The codebase lives under S8SharedCode/ and splits into two main parts:

| Component              | Path                    | Role                                          |
|------------------------|-------------------------|-----------------------------------------------|
| **Agent orchestrator** | `S8SharedCode/code/`    | Graph execution, skills, memory, sandbox      |
| **LLM Gateway V8**     | `S8SharedCode/gateway/` | FastAPI service for chat, routing, embeddings |

## Table of Contents

1. [High-Level Architecture](#high-level-architecture)
2. [How a Query Runs (End to End)](#how-a-query-runs-end-to-end)
3. [Core Modules](#core-modules)
4. [Skills System](#skills-system)
5. [Graph Growth Mechanisms](#graph-growth-mechanisms)
6. [Failure Recovery](#failure-recovery)
7. [Memory & Knowledge](#memory--knowledge)
8. [Sandbox & Code Execution](#sandbox--code-execution)
9. [LLM Gateway](#llm-gateway)
10. [Persistence & Replay](#persistence--replay)
11. [Quickstart](#quickstart)
12. [Project Layout](#project-layout)

## High-Level Architecture

```
User Query
|
?
+-----+
| flow.py - Executor                               |
| - Builds / resumes a NetworkX DiGraph           |
| - Finds "ready" nodes (all predecessors complete)|
| - Runs ready nodes in parallel (asyncio.gather)  |
| - Extends graph after each success               |
| - Handles failures via recovery.py               |
+-----+
|
+-- skills.py - render prompt, call gateway or sandbox
+-- gateway.py - bridge to LLM Gateway V8 (:8108)
```

DEADME

```
+-- memory.py - FAISS vector memory (Session 7 carryover)
+-- mcp_runner.py - multi-turn tool-use loop
+-- persistence.py - session state on disk
```

Design principle: The Planner names skills, never tools. Tool access is declared per skill in `agent_config.yaml` (`tools_allowed`). This preserves the Session 7 "tool-blindness" contract for planning.

---

## How a Query Runs (End to End)

1. **CLI entry** - `uv run python flow.py "your question"` calls `Executor.run()`.
2. **Session setup** - A session ID is created (e.g. `s8-a1b2c3d4`). The user query is saved. Memory is read once via FAISS; hits are injected into every skill prompt this run.
3. **Seed node** - A single `planner` node is added with input `USER_QUERY`.
4. **Execution loop** until no pending work:
  - \* Find **ready nodes**: status `pending` and all predecessors `complete` or `skipped`.
  - \* Mark them `running`, persist graph.
  - \* **Run in parallel** via `asyncio.gather` -> each calls `run_skill()`.
  - \* On success: store `AgentResult`, mark `complete`, call `graph.extend_from()` to splice successors.
  - \* On failure: `plan_recovery()` decides skip vs. re-plan with a new Planner node.
  - \* Persist graph and per-node JSON after each batch.
5. **Final answer** - Taken from the Formatter skill's `output.final_answer`. If no Formatter ran, the executor falls back to the last completed node's output.

Example for a simple greeting:

```
planner -> formatter
```

Example for web research:

```
planner -> researcher -> formatter
```

Example for code (student assignment):

```
planner -> coder -> sandbox_executor -> formatter
```

(`sandbox_executor` is wired automatically via `internal_successors` in `agent_config.yaml`.)

---

## Core Modules

### `flow.py` - Orchestrator

The heart of Session 8. Contains three main pieces:

| Class / function | Purpose |
|------------------|---------|
|------------------|---------|

|                                    |                                                                                |
|------------------------------------|--------------------------------------------------------------------------------|
| <code>`Graph`</code>               | NetworkX <code>`DiGraph`</code> wrapper. Nodes are IDs like <code>`n`</code> : |
| <code>`Graph.ready_nodes()`</code> | Returns nodes whose predecessors are all <code>`complete`</code>               |
| <code>`Graph.extend_from()`</code> | After a skill succeeds, adds dynamic successors, s                             |
| <code>`Executor`</code>            | Main loop: ready -> parallel run -> extend -> reco                             |
| <code>`main()`</code>              | CLI; supports <code>`--resume &lt;session_id&gt;`</code> .                     |

## ``schemas.py`` - Typed contracts

All layers communicate via Pydantic models:

- \* ``AgentResult`` - What every skill returns: ``success``, ``output`` (dict), ``successors`` (list of ``NodeSpec``), timing, errors.
- \* ``NodeSpec`` - A node to add: ``skill``, ``inputs``, ``metadata``.
- \* ``NodeState`` - Persisted per-node record including ``prompt_sent`` (exact bytes sent to the gateway).
- \* ``MemoryItem`` - FAISS-indexed memory record with optional ``embedding``.

## ``skills.py`` - Skill registry & dispatch

- \* ``SkillRegistry`` - Loads all skills from ``agent_config.yaml``.
- \* ``resolve_inputs()`` - Turns input refs (``USER_QUERY``, ``n:3``, ``art:sha...``) into prompt-ready dicts.
- \* ``render_prompt()`` - Combines skill template + query + memory hits + resolved inputs + optional failure report.
- \* ``run_skill()`` - Dispatcher:
- \* ``sandbox_executor`` -> extracts ``code`` from upstream Coder, calls ``sandbox.run_python()``.
- \* Tool-using skills -> ``mcp_runner.run_with_tools()``.
- \* Text-only skills -> single ``LLM().chat()`` call.

## ``recovery.py`` - Failure policy

| Failure type                    | Action                                                   |
|---------------------------------|----------------------------------------------------------|
| Transient (503, timeout)        | Skip - gateway already retried                           |
| Validation (malformed NodeSpec) | Skip - fix the prompt                                    |
| Upstream failure (non-Planner)  | Re-plan - new Planner with <code>`failure_report`</code> |
| Critic verdict = fail           | Skip child, splice recovery Planner (once per bran       |

## ``gateway.py`` - Gateway bridge

- \* Auto-starts LLM Gateway V8 on port ``8108`` if not running.
- \* Loads ``client.py`` from the sibling ``gateway/`` directory.
- \* Exposes ``LLM`` client and ``embed()`` helper.

## ``persistence.py`` - Session storage

Each session under `code/state/sessions/<sid>/`:

README

|                  |                                             |
|------------------|---------------------------------------------|
| query.txt        | - original user query                       |
| graph.json       | - NetworkX graph (JSON via node_link_data)  |
| nodes/n_001.json | - NodeState per node (includes prompt_sent) |

Atomic writes (tmp + rename) survive SIGKILL mid-write.

## `sandbox.py` - Python subprocess runner

Runs Coder output in a temp directory with timeout and stdout/stderr caps. Usability boundary, not security - no container isolation.

## `mcp\_runner.py` - Tool-use loop

For skills with tools\_allowed, drives multi-turn chat: model requests tool -> MCP server executes -> result fed back -> repeat until text response (max 6 hops).

---

## Skills System

Skills are defined in agent\_config.yaml + a matching prompts/<skill>.md file. There is no Python class per skill.

### Available skills

| Skill              | Tools                     | Special flags                                  | Description                                       |
|--------------------|---------------------------|------------------------------------------------|---------------------------------------------------|
| `planner`          | -                         | -                                              | Decomposes query into initial DAG; handles rec... |
| `retriever`        | `search_knowledge`        | -                                              | Searches FAISS memory index                       |
| `researcher`       | `web_search`, `fetch_url` | -                                              | Multi-step web research                           |
| `distiller`        | -                         | `critic: true`                                 | Extracts structured fields; Critic auto-inserted  |
| `summariser`       | -                         | -                                              | Condenses long content                            |
| `critic`           | -                         | -                                              | Pass/fail evaluation                              |
| `formatter`        | -                         | -                                              | Terminal node; emits `final_answer`               |
| `coder`            | -                         | `internal_successors: [sandbox_executor, ...]` | Python stub** - emits Python code                 |
| `sandbox_executor` | -                         | -                                              | Runs Coder's code in sandbox                      |
| `browser`          | -                         | -                                              | Reserved for Session 9                            |

### Skill output contract

Every LLM skill must return a single JSON object (no markdown fences). Common fields:

```
{
  "rationale": "why this output",
  "successors": [{"skill": "...", "inputs": [{"n":label}], "metadata": {"label": "..."}},
  "... skill-specific fields ..."
}
```

The Planner uses "nodes" instead of/in addition to "successors".

The Coder must emit:

README

```
{"code": "<python source>", "rationale": "<one line>"}
```

---

## Graph Growth Mechanisms

The graph grows through five mechanisms:

1. **Planner seed plan** - Initial nodes from the first Planner run.
2. **Dynamic successors** - Any skill can emit `successors` / `nodes` in its JSON output.
3. **Static internal successors** - e.g. Coder always gets `sandbox\_executor` appended via yaml.
4. **Critic auto-insertion** - Skills with `critic: true` (Distiller) get a Critic node inserted between them and their children.
5. **Recovery Planner** - On failure or Critic fail, a new Planner node is spliced in with a `failure\_report`.

### Input reference resolution

The Planner can reference sibling nodes by label:

```
{"skill": "formatter", "inputs": ["n:out"], "metadata": {"label": "out"}}
```

extend\_from() resolves n:<label> -> actual node ID (n:3) after all siblings are created.

---

## Failure Recovery

```
Node fails
|
?
classify_failure(error_text)
|
+-- transient -----? skip (no re-plan)
+-- validation_error ---? skip (fix prompt)
+-- upstream_failure
    |
    +-- failed skill == planner -? skip
    +-- else -----? new planner node with failure_report
```

Critic fail path (separate):

- \* Child node marked `skipped`
  - \* Recovery Planner queued (once per target branch)
  - \* Second fail on same branch -> cap hit, branch skipped silently (warning printed at end)
- 

## Memory & Knowledge

Carried over from Session 7:

```
***`memory.py`** - Read/write `MemoryItem` records; FAISS vector search on embeddings.
***`vector_index.py`** - FAISS index persisted under `code/state/`.
***`artifacts.py`** - Binary blob store referenced as `art:<id>`.
```

At session start, `memory.read(query)` runs once; hits appear in every skill prompt under MEMORY HITS. The Retriever skill can also call `search_knowledge` via MCP for deeper lookups.

Sample indexed papers live in `code/sandbox/papers/` (attention, CoT, ReAct, LoRA, DPO).

## Sandbox & Code Execution

Coder -> SandboxExecutor pipeline:

1. Planner emits a ``coder`` node.
2. Coder LLM returns JSON with ``"code": "..."``.
3. Orchestrator auto-adds ``sandbox_executor`` (yaml ``internal_successors``).
4. ``sandbox_executor`` extracts ``code`` from upstream output, calls ``sandbox.run_python()``.
5. Returns stdout, stderr, exit code, files written, timeout flag.

The student assignment is to implement `prompts/coder.md` so the Coder reliably emits valid Python in the required JSON shape.

## LLM Gateway

The gateway (`S8SharedCode/gateway/`) is a FastAPI service on port 8108 that:

- \* Routes chat requests across multiple LLM providers (Gemini, Groq, Ollama, etc.)
- \* Auto-routes by cognitive layer (``perception``, ``memory``, ``decision``) and token tier (TINY / LARGE / HUGE)
- \* Provides embeddings via ``POST /v1/embed`` (768-dim, Ollama ``nomic-embed-text`` + Gemini fallback)
- \* Logs cost-by-agent when skills pass ``agent=<skill_name>``
- \* Retries once on 5xx/timeout

The agent treats the gateway as an external service - start it with:

```
cd S8SharedCode/gateway && uv run main.py
```

Or let `flow.py` auto-launch it via `ensure_gateway()`.

## Persistence & Replay

Inspect a session:

```
cd S8SharedCode/code
uv run python replay.py <session_id>
```

Replay walks nodes/n\_\*.json in completion order and shows each skill's status, inputs, output, and prompt\_sent (the exact prompt that hit the gateway).

Resume after interrupt:

```
uv run python flow.py --resume <session_id>
```

Running nodes are reset to pending on resume.

## Quickstart

Prerequisites: Python 3.11+, [uv](https://docs.astral.sh/uv/), Ollama with nomic-embed-text, at least one LLM API key.

```
# 1. Configure secrets
cp S8SharedCode/.env.example S8SharedCode/.env
# Edit .env with your API keys

# 2. Install dependencies
cd S8SharedCode/gateway && uv sync && cd ../..
cd S8SharedCode/code && uv sync && cd ../..

# 3. Start gateway (terminal 1)
cd S8SharedCode/gateway && uv run main.py

# 4. Run agent (terminal 2)
cd S8SharedCode/code
uv run python flow.py "hello"
```

A successful run prints Planner + Formatter node lines and a final answer. Sessions are saved under code/state/sessions/.

## Project Layout

```
Assignment 8/
+-- README.md          <- this file
+-- CODE_EXPLANATION.pdf <- PDF version of this documentation
|
+-- S8SharedCode/
|   +-- README.md      <- course quickstart (assignment-focused)
|   +-- .env.example
|   |
|   +-- code/          <- agent orchestrator
|   |   +-- flow.py    <- start here: Graph + Executor + CLI
|   |   +-- skills.py  <- skill registry, prompt rendering, dispatch
|   |   +-- recovery.py <- failure classification + critic handling
|   |   +-- persistence.py <- session read/write
|   |   +-- sandbox.py  <- subprocess Python runner
```

| README                |                                           |
|-----------------------|-------------------------------------------|
| +-- mcp_runner.py     | <- tool-use loop                          |
| +-- gateway.py        | <- bridge to LLM Gateway V8               |
| +-- schemas.py        | <- Pydantic models                        |
| +-- agent_config.yaml | <- skill catalogue                        |
| +-- memory.py         | <- FAISS memory (S7 carryover)            |
| +-- vector_index.py   |                                           |
| +-- artifacts.py      |                                           |
| +-- mcp_server.py     | <- MCP tools (web_search, fetch_url, ...) |
| +-- replay.py         | <- session trace viewer                   |
| +-- prompts/          | <- one .md system prompt per skill        |
| +-- tests/            |                                           |
| +-- state/            | <- runtime data (sessions, memory, FAISS) |
|                       |                                           |
| +-- gateway/          | <- LLM Gateway V8 (FastAPI, port 8108)    |
| +-- main.py           |                                           |
| +-- client.py         |                                           |
| +-- router.py         |                                           |
| +-- providers.py      |                                           |
| +-- embedders.py      |                                           |
| +-- static/           | <- dashboard + help pages                 |

## Troubleshooting

| Symptom                                         | Likely cause                                             | Fix                                                                         |
|-------------------------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------------|
| Gateway failed to start within 45s              | Gateway not running or port 8108 taken                   | Start gateway manually; check <code>.env` keys</code>                       |
| 503 Service Unavailable                         | All providers in cooldown                                | Add another API key or wait                                                 |
| <code>`no code in upstream coder output`</code> | Coder prompt not emitting <code>`{"code": "..."}`</code> | Implement <code>`prompts/coder.md`</code> per spec                          |
| Thin/wrong final answer                         | Bad plan or failed branch                                | <code>`replay.py &lt;sid&gt;`</code> and inspect <code>`prompt_sent`</code> |

## What Not to Modify (Course Scope)

- \* Session 7 carryover modules: ``perception.py``, ``decision.py``, ``action.py``, ``memory.py``, ``vector_index.py``, ``artifacts.py``, ``mcp_server.py``
- \* Gateway internals (treat as a service; report bugs separately)

Generated documentation for EAGV3 Session 8 - Multi-Agent Growing-Graph Orchestrator.

## Assignment Evidence Log

How to run + full status: [ASSIGNMENT\_STATUS.md](./ASSIGNMENT\_STATUS.md)

Submission one-pager: [SUBMISSION.md](./SUBMISSION.md)

This section records concrete runs and session IDs for Session 8 deliverables.



## 1) Base Queries (hello, A, I, J, K)

```
* `hello`
* Session: `s8-7f8a75bf`
* Result: `Hello!`
* `A` (Claude Shannon extract)
* Session: `s8-dc7d6377`
* Graph shape: planner -> researcher -> distiller -> formatter
* `I` (London/Paris/Berlin populations)
* Session: `s8-30cc4b2c`
* Graph shape: planner -> 3x researcher (parallel) -> coder -> formatter + sandbox_executor
* `J` (graceful fail path)
* Session: `s8-ea8e55ff`
* Result: user-facing unavailable-path explanation, no crash
* `K` (kill and resume)
* Interrupted session: `s8-65c8069d`
* Resume command: `uv run python flow.py --resume s8-65c8069d`
* Resumed completion observed
```

## 2) Custom Parallel Fan-out Query

Run (Part 2):

```
cd S8SharedCode/code
./scripts/run_assignment.sh 2
# or:
uv run python flow.py "Run three parallel research branches: (1) From
https://en.wikipedia.org/wiki/CRISPR - one sentence on what CRISPR is. (2) From
https://en.wikipedia.org/wiki/mRNA_vaccine - one sentence on how mRNA vaccines work. (3) From
https://en.wikipedia.org/wiki/Photovoltaic_effect - one sentence on how solar cells convert light.
Then merge the three answers and say which two topics are most related to medicine."
```

Query: three independent Wikipedia URLs -> Planner emits 3x researcher concurrent nodes (CRISPR, mRNA vaccine, photovoltaic effect), then merge -> formatter.

Session: `_run` and paste `s8-*` here\_

Verify: `uv run python scripts/verify_parallel_timing.py <sid>` -> expect PASS (wall ? max branch, not sum).

Alternate logged run (4 cities): `s8-14af4aa5` - Tokyo/Delhi/Shanghai/São Paulo populations.

Why it fan-outs (city alternate): Four named cities -> four researcher nodes (`r_tokyo`, ...), then distiller.

## 3) Critic Pass + Fail + Recovery

Design: `[S8SharedCode/code/CRITIC_VERDICT.md](S8SharedCode/code/CRITIC_VERDICT.md)` - explicit critic verifies coder JSON sum matches `23+19` (no tools; in-prompt arithmetic).

Run: `./scripts/run_assignment.sh 3`

| Run                 | Query                                         | Session         |
|---------------------|-----------------------------------------------|-----------------|
| **Pass**            | `Use Python to compute 23 plus 19. The coder` | `s8-1dc4d467a0` |
| **Fail + recovery** | `Use Python to compute 23 plus 19. Emit JSON` | `s8-f3e362d0`   |

Graph shape: coder -> critic -> formatter

Fail expect: critic-fail recovery: planner node ... then corrected sum 42.

#### 4) Coder Skill Implementation + Sandbox Validation

- \* `coder.md` stub replaced with strict JSON output contract:
- \* `{ "code": "<python>", "rationale": "..."} `
- \* Manual chain validation (from `skills.run\_skill`):
- \* `coder\_success=True`
- \* `sandbox\_success=True`
- \* `stdout: The sum of 123 and 456 is: 579`

#### 5) New Skill Added

- \* Added skill: `comparator`
- \* Config: `S8SharedCode/code/agent\_config.yaml`
- \* Prompt: `S8SharedCode/code/prompts/comparator.md`
- \* Planner updated to route ranking/closest tasks via comparator.
- \* Validation query sessions:
- \* `s8-1dc4d047` - first comparator graph (weak researcher outputs)
  - \* `s8-a6972d7c` - \*\*re-run\*\* with real Madrid/Rome figures; sandbox recovery splice; view `state/sessions/s8-a6972d7c/graph\_view.html`

#### Regression Safety

- \* Recovery tests: `uv run pytest tests/test\_recovery.py`
- \* Result: `22 passed`
- \* Final smoke run:
- \* Session: `s8-f3e362d0`
- \* Result: `Hello!`

## YouTube Demo Checklist

Pre-generated DAG HTML for key sessions: S8SharedCode/code/state/sessions/<sid>/graph\_view.html

Regenerate: cd S8SharedCode/code && uv run python visualize\_graph.py <sid> --open

- \* [ ] Show environment startup (`uv sync`, gateway up, `flow.py` hello)
- \* [ ] Show Coder prompt implementation file and one coder+sandbox run
- \* [ ] Show all 5 base query runs with session IDs
- \* [ ] Show custom parallel fan-out run and explain barrier/merge

- \* [ ] Show Critic pass run and fail+recovery splice run<sup>README</sup>
- \* [ ] Show new `comparator` skill config + prompt + execution run
- \* [ ] Show `pytest tests/test\_recovery.py` passing